

Various approaches for encoding model

July 2, 2026

1 From Meta's Submission

This was one of the standout tricks: modality dropout during training forced the model to remain robust even when a modality was missing, which paid off directly on the OOD set since it included a silent black-and-white Charlie Chaplin film with no audio track.

Also here for creating time bucket here they just added embeddings lying in that time frame which can be improved.

The brain surface used in Algonauts divides brain into 1000 parcels. Meta didn't train just one model — they trained several model variants (different architectures/hyperparameter configs, or the same architecture across different training runs). Each variant produces its own prediction for every single parcel (personally I don't know whether we have that much time to incorporate this but it appeared excellent approach) .

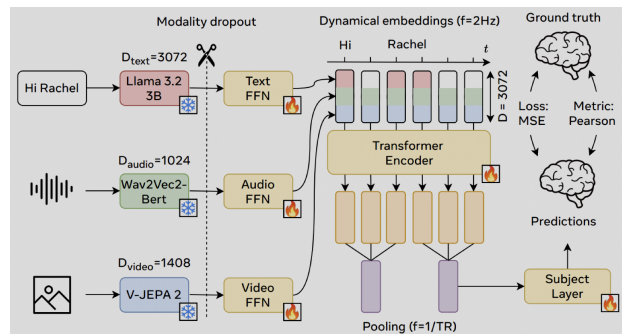


Figure 1: Tribe Architecture

2 From VIBE’s Submission

The unique thing in here was- they used multimodel architecture for feature extraction from each modality because we can’t expect single model to capture everything (we can definitely try incorporating this with meta’s encoder model).

- **VISION-** V-JEPA 2 (ViT-L/16), SlowFast R101, Qwen2.5-Omni 3B
- **AUDIO** - BEATs, Whisper V3 encoder
- **TEXT** - Qwen2.5 14B

This is VIBE’s real signature — splitting fusion and temporal modeling into two separate transformers rather than doing both at once like TRIBE:

Stage 1 — Modality Fusion Transformer: every feature stream is first projected down to 256 dimensions via a simple feedforward layer, and a 256-dim subject embedding is included alongside them. Critically, this transformer is single-layer and does not model time at all — it does cross-attention independently within each TR, fusing only same-timestep features across modalities. The output is one fused vector per TR.

Stage 2 — Prediction Transformer: a two-layer transformer that models dependencies across TRs, analogous to how an LLM processes a token sequence, with a final feedforward layer producing the fMRI prediction.

Positional embeddings — RoPE beat everything else tried: fixed sinusoidal embeddings actually hurt performance, and simple concatenation of position info didn’t help either; switching to Rotary Positional Embeddings, which encode relative rather than absolute position, gave a significant boost.

3 RNN approach from winner

3.1 Feature Extraction

- **Video (4 models):** SlowFast, VideoMAE, Swin Transformer, CLIP — each captures something different (motion, general video understanding, hierarchical spatial features, and image-text alignment respectively).
- **Audio (3 models):** HuBERT, WavLM (general self-supervised speech/audio), and CLAP (semantic audio-text embeddings).

- **Text (2 models):** BERT (short local context) and Longformer (long-range context, since it can handle much longer documents than BERT). To avoid a "cold start" problem at the beginning of each episode's transcript, they even prepended the previous episode's dialogue when extracting features — like giving the model a quick recap before the episode starts.

3.2 Architecture — three stages, RNN-based

1. **Per-modality bidirectional RNNs:** each modality's feature sequence goes through its own LSTM that reads both forward and backward in time (bidirectional — it can use a bit of "future" context too, similar to VIBE's no-causal-mask trick, just via a different mechanism).
2. **Fusion by simple averaging:** unlike VIBE's learned fusion transformer, they just averaged the modality hidden states together element-by-element. They explicitly tested fancier fusion (learned weights, attention) and found plain averaging worked just as well and acted as a regularizer (helped prevent overfitting).
3. **Second RNN on the fused sequence:** models how the combined signal evolves over time — same conceptual role as VIBE's "prediction transformer," just RNN-based instead.
4. **Subject-specific output heads:** instead of a subject embedding fed into a shared model (like TRIBE), they keep all the shared RNN layers common across subjects but give each subject their own final linear layer. Efficient — you share the "understanding" but let each person's brain-to-prediction mapping be different.

3.3 Data Cleaning

Before finalizing, they trained a model, used it to score every training segment, and threw out the segments where predictions were essentially garbage (near-zero correlation) — treating these as likely noisy/bad recordings rather than something to keep forcing the model to fit.